Методичні вказівки до лабораторної роботи №1

Тема: налаштування робочого середовища та підготовка вибірки

Документ містить постановку задачі повністю і описує порядок виконання: що робити на кожному кроці, якими засобами і як перевірити результат. Вимоги до кожної вибірки наведені на початку її розділу, вимоги до звіту - у розділі 10. Готових розв'язків тут немає, реалізація лишається частиною роботи. Команди стосуються налаштування середовища, фрагменти коду - перевірки вже отриманого результату.

Розділи з позначкою **(опціонально)** на оцінку не впливають. Це самоперевірки і поради: вони допомагають знайти помилку раніше, ніж її знайде викладач, але постановка задачі їх не вимагає.

---

0. Постановка задачі

Налаштувати середовище програмування Python/Jupyter, підготувати та візуалізувати вибірки даних для подальшого виконання алгоритмів кластеризації у просторі JupyterLab. Використання алгоритмів кластеризації не є предметом цієї лабораторної роботи.

Задача - отримати чотири вибірки, придатні до кластеризації в ЛР2-ЛР4, і переконатися, що середовище працює.

Що має вийти в результаті

Артефакт	Де
Проєкт з відтворюваним оточенням	`pyproject.toml` + `uv.lock`
Ноутбук з усім ходом виконання	`notebooks/lab1.ipynb`
Чотири нормалізовані вибірки у csv	`data/sample1.csv` … `sample4.csv`
Графіки всіх чотирьох вибірок	у ноутбуці та `figures/`
Звіт	`.doc`, `.docx`, `.odt` або `.pdf`

Вибірки нумеруються наскрізно: ЛР2, ЛР3 і ЛР4 працюють з тими самими чотирма файлами. Тому csv краще одразу покласти в теку, яку не доведеться переносити.

Два шляхи налаштування середовища. Розділи 1-3 описують локальне середовище на `uv` - основний варіант. Розділ 3A описує Google Colab - альтернатива, якщо локальне встановлення неможливе. Розділи 4-12 однакові для обох шляхів.

Постановка задачі вимагає Python 3.10 і JupyterLab, а не конкретний менеджер пакетів. `uv` рекомендований тим, що сам ставить потрібний інтерпретатор і фіксує версії залежностей, але `venv` з `pip`, conda чи будь-яке інше оточення з пакетами з розділу 2.1 підходять так само. Змінюються тільки команди розділів 1-2, решта роботи від цього не залежить.

---

1. Встановлення uv

Постановка задачі:

- Встановити Python 3.10
- Встановити JupyterLab

`uv` - менеджер пакетів і версій Python. Він сам завантажує потрібний інтерпретатор, тому Python 3.10 окремо ставити не треба, а JupyterLab встановлюється разом з рештою пакетів у розділі 2.

Windows (PowerShell):

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

або, якщо є WinGet:

```
winget install --id=astral-sh.uv -e
```

Linux / macOS:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Після встановлення **закрити і відкрити термінал заново** - інакше `uv` не знайдеться в `PATH`. Перевірка:

```
uv --version
```

Очікуваний вигляд відповіді: `uv 0.9.18 (0cee76417 2025-12-16)`. Конкретна версія значення не має, важливо, що команда відпрацювала без помилки.

---

2. Створення проєкту

```
uv init iad-lab1 --python 3.10
cd iad-lab1
```

> **Типова помилка.** Якщо виконати просто `uv init iad-lab1`, а потім `uv python pin 3.10`, буде помилка: `error: The requested Python version 3.10 is incompatible with the project requires-python value of >=3.13`. Причина: `uv init` без ключа записує в `pyproject.toml` ту версію, яка стоїть у системі. Прапорець `--python 3.10` задає версію одразу і правильно.

Перевірити, що вийшло:

```
cat pyproject.toml        # requires-python = ">=3.10"
cat .python-version       # 3.10
```

Додати потрібні пакети однією командою:

```
uv add jupyterlab drawdata pandas numpy matplotlib scikit-learn openpyxl
```

> **Типова помилка.** Додавати або перевстановлювати пакети командою `uv pip install`. У проєкті на `uv` пакети додаються тільки через `uv add`: вона вносить пакет у `pyproject.toml`, перераховує `uv.lock` і лише після цього змінює `.venv`. `uv pip install` змінює `.venv` повз ці два файли, тому оточення перестає відповідати замку. Наслідків два: проєкт більше не відтворюється з архіву командою `uv sync` у когось іншого, і всередині `.venv` легко опиняються несумісні між собою версії - саме звідси беруться помилки віджетів з розділу 5. Стан оточення повертає команда `uv sync`: вона приводить `.venv` точно до `uv.lock`.

2.1. Що робить кожен пакет

Сім пакетів, кожен закриває конкретний пункт постановки задачі.

`jupyterlab` - середовище, у якому виконується робота. Дає браузерний редактор ноутбуків: код розбивається на клітинки, кожна виконується окремо, а результат (число, таблиця, графік, інтерактивний віджет) лишається під нею. Для цієї роботи це принципово: Вибірка 1 малюється мишею в інтерактивному віджеті, чого звичайний `.py`-скрипт не вміє. Разом з ним ставиться ядро, яке власне виконує код Python.

`pandas` - таблиці з іменованими колонками (`DataFrame`). Уся робота з даними тримається на ньому: `read_csv` і `to_csv` для вибірок, `read_excel` для xlsx-варіанта ірисів, `factorize` для перетворення текстової мітки на цілий номер кластера, `quantile` для меж викидів, `dropna` і `fillna` для пропусків, `clip` для обрізання, `describe` для огляду набору, `corr` для матриці кореляцій, `plotting.scatter_matrix` для матриці попарних проєкцій і `plotting.parallel_coordinates` для паралельних координат.

`numpy` - масиви чисел і операції над ними. У роботі потрібен насамперед для Вибірки 2: генератор випадкових чисел `default_rng` і його методи `uniform` (центри кластерів, значення дисперсії) та `normal` (точки кластера за нормальним розподілом). Також `linalg.norm` для відстаней у перевірці перетину. Pandas побудований поверх numpy, тому він все одно встановиться, але тут він використовується напряму.

`matplotlib` - побудова графіків. Двовимірні розсіювання для Вибірки 1, тривимірне для Вибірки 2 (вісь з `projection="3d"`), збереження у png через `savefig`. Pandas малює матрицю розсіювання теж через matplotlib.

`scikit-learn` - бібліотека машинного навчання. У цій роботі з неї беруться дві речі: `preprocessing.MinMaxScaler` для нормалізації ознак у відрізок [0, 1] і `datasets.fetch_openml` для завантаження відкритих наборів у Вибірці 4. Опціонально - `decomposition.PCA` для проєкції багатовимірної вибірки на площину (розділ 4.1). Алгоритми кластеризації з неї знадобляться лише в ЛР2, тут вони не потрібні.

`drawdata` - Jupyter-віджети для малювання даних мишею. Потрібен рівно для одного пункту: `ScatterWidget` у Вибірці 1 дає полотно з палітрою кольорів, а властивість `data_as_pandas` віддає намальоване як таблицю з колонками `x`, `y`, `color`. Прямо названий у постановці задачі.

`openpyxl` - читання і запис файлів `.xlsx`. Сам pandas формат Excel не розбирає, він делегує це стороннім бібліотекам, тому без `openpyxl` виклик `read_excel` впаде з повідомленням `Missing optional dependency 'openpyxl'`. Потрібен тільки тим, хто візьме іриси у форматі xlsx замість csv.

`uv` створить `.venv/` і `uv.lock`. Вручну активовувати оточення не треба: будь-яка команда запускається через `uv run`.

Створити теки під результати:

```
mkdir data figures notebooks
```

Структура проєкту на цьому кроці:

```
iad-lab1/
├── .python-version
├── pyproject.toml
├── uv.lock
├── data/
├── figures/
└── notebooks/
```

Файл `дані/iris.csv` з комплекту лабораторних скопіювати в `data/` проєкту - він знадобиться для Вибірки 3.

---

3. Запуск JupyterLab

З теки проєкту:

```
uv sync
uv run jupyter lab
```

`uv sync` приводить `.venv` до того, що записано в `uv.lock`. Одразу після розділу 2 оточення вже зібране, тому команда відпрацює за мить і нічого не змінить. Обов'язковою вона стає, коли в теці немає `.venv`: після перенесення проєкту на іншу машину, розпакування з архіву або запуску еталонного проєкту `example/iad-lab1` з комплекту лабораторних.

У вікні термінала з'явиться рядок виду `http://localhost:8888/lab?token=...`. Браузер відкриється сам; якщо ні - відкрити це посилання вручну.

Створити ноутбук `notebooks/lab1.ipynb` і одразу перевірити оточення першою клітинкою:

```
import sys, pandas, numpy, matplotlib, sklearn, drawdata
print(sys.version)
print("pandas", pandas.__version__, "| numpy", numpy.__version__)
print("sklearn", sklearn.__version__, "| matplotlib", matplotlib.__version__)
```

Очікується `3.10.x` та номери версій без жодного `ModuleNotFoundError`.

> **Типова помилка.** `ModuleNotFoundError: No module named 'drawdata'` при тому, що `uv add drawdata` пройшов успішно. Причина майже завжди одна - ноутбук підключився до чужого ядра. У списку ядер поруч з ядром проєкту можуть бути старі глобальні (`py310`, `Python 3.12` тощо). Треба: 1. запускати JupyterLab саме командою `uv run jupyter lab` з теки проєкту; 2. у правому верхньому куті ноутбука вибрати ядро `Python 3 (ipykernel)`; 3. переконатися, що `sys.executable` вказує всередину `.venv` проєкту: `python import sys; print(sys.executable)`

3.1. Перші клітинки: приклади, що мають запрацювати

П'ять коротких прикладів. Дані в них вигадані на місці, до вибірок роботи вони не мають стосунку - це перевірка, що середовище живе, і заразом зразки викликів, які знадобляться далі. Кожен виконується в окремій клітинці.

Приклад 1. Hello world і де саме виконується код.

```
print("Hello, JupyterLab!")

import sys
print(sys.version)
print("executable:", sys.executable)
```

Вивід:

```
Hello, JupyterLab!
3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)]
executable: ...\iad-lab1\.venv\Scripts\python.exe
```

Шлях в `executable` має вести всередину `.venv` проєкту. Якщо веде кудись інде - ноутбук на чужому ядрі, див. попередню врізку.

Приклад 2. Найпростіше розсіювання.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)
x = rng.normal(0, 1, 200)
y = 2 * x + rng.normal(0, 0.5, 200)

fig, ax = plt.subplots(figsize=(5, 4))
ax.scatter(x, y, s=10)
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Перевірка matplotlib")
plt.show()
```

Під клітинкою має з'явитися картинка з витягнутою по діагоналі хмарою з 200 точок. Якщо картинки немає, а є рядок виду `<Figure size 500x400 with 1 Axes>` - не виконано `plt.show()` або фігура не є останнім виразом клітинки.

Приклад 3. Розфарбування за міткою, легенда і збереження у файл.

Саме цей шаблон повторюється для Вибірок 1, 3 і 4.

```
import pandas as pd
from pathlib import Path

FIG = Path("../figures")
FIG.mkdir(parents=True, exist_ok=True)

rng = np.random.default_rng(1)
demo = pd.concat(
    [
        pd.DataFrame({"x": rng.normal(cx, 0.6, 150),
                      "y": rng.normal(cy, 0.6, 150),
                      "cluster": k})
        for k, (cx, cy) in enumerate([(0.0, 0.0), (4.0, 1.0), (2.0, 4.0)])
    ],
    ignore_index=True,
)

fig, ax = plt.subplots(figsize=(6, 5))
for value, group in demo.groupby("cluster"):
    ax.scatter(group["x"], group["y"], s=10, label=f"кластер {value}")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Три кластери, розфарбовані за міткою")
ax.legend()
fig.savefig(FIG / "demo.png", dpi=120, bbox_inches="tight")

print(demo.shape)
print(demo["cluster"].value_counts().sort_index().to_dict())
```

Вивід:

```
(450, 3)
{0: 150, 1: 150, 2: 150}
```

Три різнокольорові згустки на графіку, легенда з трьома підписами і новий файл `figures/demo.png`. Розфарбування робиться циклом по `groupby`, а не однією командою: кожен виклик `scatter` додає окремий шар зі своїм підписом, і тільки тому легенда взагалі з'являється.

Приклад 4. Тривимірне розсіювання.

Потрібне для Вибірки 2, у якої три координати.

```
rng = np.random.default_rng(2)
demo3 = pd.concat(
    [
        pd.DataFrame(rng.normal(c, 0.6, size=(150, 3)),
                     columns=["x", "y", "z"]).assign(cluster=k)
        for k, c in enumerate([(0.0, 0.0, 0.0), (4.0, 1.0, 2.0), (2.0, 4.0, -1.0)])
    ],
    ignore_index=True,
)

fig = plt.figure(figsize=(6, 5))
ax = fig.add_subplot(projection="3d")
for value, group in demo3.groupby("cluster"):
    ax.scatter(group["x"], group["y"], group["z"], s=8, label=f"кластер {value}")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("z")
ax.legend()
fig.savefig(FIG / "demo3d.png", dpi=120, bbox_inches="tight")
```

Відмінність від прикладу 3 в одному рядку: вісь створюється через `fig.add_subplot(projection="3d")`, після чого `scatter` приймає три масиви замість двох. Окремий пакет для цього не потрібен, підтримка 3d є в matplotlib.

Приклад 5. Читання csv у таблицю.

```
iris = pd.read_csv("../data/iris.csv")
print(iris.shape)
iris.head()
```

Вивід:

```
(150, 5)
   Sepal_length  Sepal_width  Petal_length  Petal_width      Species
0           5.1          3.5           1.4          0.2  Iris-setosa
1           4.9          3.0           1.4          0.2  Iris-setosa
2           4.7          3.2           1.3          0.2  Iris-setosa
3           4.6          3.1           1.5          0.2  Iris-setosa
4           5.0          3.6           1.4          0.2  Iris-setosa
```

`head()` виведено останнім виразом клітинки без `print`, тому JupyterLab малює таблицю, а не текст. Якщо замість таблиці помилка `FileNotFoundError` - файл `iris.csv` не скопійовано в `data/` проєкту, див. кінець розділу 2.

3A. Альтернатива: Google Colab

Colab дає готове середовище в браузері: нічого не встановлюється, Python і більшість пакетів уже на місці. Підходить, якщо на машині немає прав на встановлення програм або бракує місця.

Чим доводиться платити. Середовище зникає при відключенні (зазвичай через 90 хвилин бездіяльності або 12 годин роботи). Усе, що не збережено на Google Drive, зникає разом з ним. Тому крок з підключенням Drive пропускати не можна.

3А.1. Створення ноутбука

1. Відкрити https://colab.research.google.com і увійти в обліковий запис Google.
2. `Файл -> Створити блокнот`.
3. Перейменувати з `Untitled0.ipynb` на `lab1.ipynb` - клацнути на назві вгорі.

3А.2. Підключення Google Drive

Перша клітинка ноутбука:

```
from google.colab import drive
drive.mount('/content/drive')
```

Після запуску Colab запитає дозвіл. Треба натиснути `Підключитися до Google Drive`, вибрати обліковий запис і підтвердити доступ. Успішна відповідь: `Mounted at /content/drive`.

Робочі теки створюються на Drive, у `/content/drive/MyDrive/iad-lab1/`, з підтеками `data` і `figures` (`Path.mkdir(parents=True, exist_ok=True)`). Тоді все записане в них лишається після зникнення середовища. Далі в розділах 4-9 замість `Path("../data")` використовуються ці шляхи, решта роботи не змінюється.

3А.3. Перевірка версій

```
import sys, pandas, numpy, matplotlib, sklearn
print(sys.version)
print("pandas", pandas.__version__, "| numpy", numpy.__version__)
print("sklearn", sklearn.__version__, "| matplotlib", matplotlib.__version__)
```

Фактичний вивід Colab станом на вересень 2026:

```
3.13.15 (main, Aug  6 2026, 11:06:22) [GCC 13.3.0]
pandas 2.2.3 | numpy 2.1.3
sklearn 1.6.1 | matplotlib 3.10.0
```

`pandas`, `numpy`, `matplotlib`, `scikit-learn` у Colab уже встановлені. Версія Python новіша за 3.10, і змінити її штатними засобами не можна. Увесь код роботи сумісний з нею, тому у звіті вказується та версія, яка фактично виведена.

3А.4. Встановлення drawdata

Це єдиний пакет, якого в Colab немає:

```
!pip install -q drawdata
```

Після встановлення **обов'язково увімкнути менеджер віджетів** - без цього віджет `ScatterWidget` не відобразиться, і замість нього буде порожнє місце або текстовий рядок виду `ScatterWidget(...)`:

```
from google.colab import output
output.enable_custom_widget_manager()
```

> **Типова помилка.** Порядок має значення. `enable_custom_widget_manager()` викликається до створення віджета. Якщо віджет уже створено, а віджети не увімкнені - виконати клітинку з `output.enable_custom_widget_manager()`, а потім перевиконати клітинку з `ScatterWidget`.

3A.5. Завантаження iris.csv

Варіант 1 - покласти файл на Drive вручну через веб-інтерфейс і читати за шляхом `DATA / 'iris.csv'`.

Варіант 2 - завантажити з машини прямо в ноутбук через `files.upload()`:

```
from google.colab import files
uploaded = files.upload()          # відкриє діалог вибору файлу
```

Файл потрапить у `/content/`, звідки його треба одразу скопіювати на Drive (`shutil.copy`), інакше він зникне разом із середовищем.

3A.6. Що здавати з Colab

Ноутбук вивантажується через `Файл` -> `Завантажити` -> `Завантажити .ipynb`. Вибірки і графіки беруться з теки `iad-lab1` на Google Drive.

`pyproject.toml` і `uv.lock` у цьому варіанті відсутні. Замість них у звіті наводиться список версій з розділу 3A.3 і рядок `!pip install drawdata`.

---

4. Спільна підготовка

Дві допоміжні функції заводяться один раз на початку ноутбука, щоб не дублювати їхній код у чотирьох місцях:

Функція	Що робить	Чим користуватися
нормалізація	приводить перелічені ознаки до [0, 1], не чіпаючи мітку	`sklearn.preprocessing.MinMaxScaler`, метод `fit_transform`
візуалізація	розсіювання з розфарбуванням за міткою і збереженням у `figures/`	`matplotlib.pyplot`, `DataFrame.groupby`, `Axes.scatter`, `Figure.savefig`

Шляхи `DATA` і `FIG` теж задаються один раз (`pathlib.Path`), бо ноутбук лежить у `notebooks/`, а результати - на рівень вище.

Чому саме min-max, а не z-score. Наступні роботи рахують Евклідову відстань. Якщо одна ознака вимірюється в сантиметрах (0-8), а інша в гривнях (0-50000), то відстань визначатиметься виключно другою ознакою, і кластеризація зведеться до сортування за нею. Min-max приводить кожну ознаку до відрізка [0, 1], тобто вирівнює їхній внесок у відстань. Z-score теж прийнятний, але дає необмежений діапазон, через що графіки вибірок стають несумісними між собою.

Нормалізувати треба тільки ознаки. Номер кластера - мітка. Якщо пропустити його через `MinMaxScaler`, кластери 0, 1, 2 перетворяться на 0.0, 0.5, 1.0, і в ЛР2 порівняння з відомим значенням зламається. Тому список ознак у функцію передається явно, без добору через `select_dtypes`.

Перевірка після кожної нормалізації:

```
print(df[features].min().round(3).to_dict())   # yci 0.0
print(df[features].max().round(3).to_dict())   # yci 1.0
```

4.1. Графіки для вибірок з трьома і більше ознаками

Площина показує рівно дві ознаки. Вибірки 2, 3 і 4 мають три, чотири і 5+ ознак, тому одне розсіювання x-y для них - це мовчазний вибір двох ознак з відкиданням решти. Пара кластерів, розділена лише третьою ознакою, на такому графіку зіллється в одну хмару, і у звіті з'явиться висновок, якого дані не підтверджують.

Ознак	Основний графік	Чим доповнити
2	розсіювання, `Axes.scatter`	-
3	тривимірне розсіювання	матриця розсіювання 3×3
4-8	матриця розсіювання, `pandas.plotting.scatter_matrix`	паралельні координати
понад 8	матриця по 4-5 відібраних ознаках	теплокарта кореляцій для відбору, паралельні координати або проєкція PCA для загальної картини

Далі наведені всі чотири графіки з таблиці. Приклади виміряні на ірисах (`sample3` , список ознак `features3`) і на наборі `wine-quality-red` (`sample4` , одинадцять ознак у `features4`); `FIG` і нормалізація - з розділу 4.

Матриця розсіювання - основний варіант для чотирьох і більше ознак.

```
from matplotlib.lines import Line2D
from pandas.plotting import scatter_matrix

palette = ["tab:blue", "tab:orange", "tab:green", "tab:red", "tab:purple"]

axes = scatter_matrix(
    sample3[features3],
    c=[palette[k] for k in sample3["cluster"]],
    s=12,
    alpha=0.8,
    diagonal="hist",
    figsize=(8, 8),
)

fig = axes[0, 0].get_figure()
fig.legend(
    handles=[Line2D([], [], marker="o", linestyle="", color=palette[k],
                    label=f"кластер {k}")
             for k in sorted(sample3["cluster"].unique())],
    loc="upper right",
)
fig.savefig(FIG / "sample3_matrix.png", dpi=120, bbox_inches="tight")
```

N ознак дають сітку N × N: на діагоналі гістограма кожної ознаки, решта клітинок - усі пари, продубльовані дзеркально відносно діагоналі. Параметр `c=` приймає список кольорів завдовжки з вибірку, тому мітка перетворюється на колір індексуванням палітри. Легенди `scatter_matrix` не

будує - її збирають вручну з маркерів `Line2D`, інакше на графіку немає підказки, який колір якому кластеру відповідає.

На ірисах з матриці видно те, чого не дає жодна окрема пара: кластер 0 (`Iris-setosa`) відокремлюється в кожній клітинці, де бере участь довжина або ширина пелюстки, а `versicolor` і `virginica` перекриваються скрізь. Це вже відповідь на питання, наскільки вибірка легка для ЛР2.

Межа методу - кількість клітинок росте як N^2. Чотири ознаки дають 16 клітинок на полотні 8×8 дюймів і читаються, одинадцять дають 121 клітинку розміром близько 50 пікселів, де не видно ні точок, ні підписів. Тому для Вибірки 4 ознаки спочатку відбираються (розділ 8.4), а матриця будується по 4-5 з них.

Паралельні координати - усі ознаки на одному полотні.

```
from pandas.plotting import parallel_coordinates

fig, ax = plt.subplots(figsize=(10, 4))
parallel_coordinates(
    sample3[features3 + ["cluster"]],
    "cluster",
    ax=ax,
    color=palette[:sample3["cluster"].nunique()],
    alpha=0.3,
)
ax.set_ylim(0, 1)
ax.tick_params(axis="x", labelrotation=90, labelsize=8)
fig.savefig(FIG / "sample3_parallel.png", dpi=120, bbox_inches="tight")
```

Кожен рядок вибірки - ламана через усі осі-ознаки, колір - за міткою. Кластер читається як джгут ліній, що тримається разом; ознака, на якій джгути розходяться, є інформативною. На ірисах лінії кластера 0 відриваються від решти рівно на осях `Petal_length` і `Petal_width`, а на `Sepal_width` усі три кольори перемішані.

Дві умови застосовності. Ознаки мають бути нормалізовані, інакше вісь з найбільшим діапазоном притисне решту до нуля - у цій роботі нормалізація вже зроблена. Рядків має бути небагато: 1599 рядків Вибірки 4 навіть при `alpha=0.1` зливаються в суцільну заливку, тому береться підвибірка `sample4.sample(200, random_state=0)`.

Теплокарта кореляцій - інструмент відбору, а не картина кластерів.

```
corr = sample4[features4].corr()

fig, ax = plt.subplots(figsize=(7, 6))
im = ax.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)
ax.set_xticks(range(len(features4)), features4, rotation=90, fontsize=8)
ax.set_yticks(range(len(features4)), features4, fontsize=8)
fig.colorbar(im, ax=ax, shrink=0.8)
fig.savefig(FIG / "sample4_corr.png", dpi=120, bbox_inches="tight")
```

Дві ознаки з модулем кореляції, близьким до одиниці, несуть той самий сигнал, і друга з них займе клітинку в матриці, нічого не додавши. Вимірювання на `wine-quality-red`: чотири ознаки з найбільшим розкидом - `citric_acid` (0.195), `alcohol` (0.164), `fixed_acidity` (0.154), `free_sulfur_dioxide` (0.147), але кореляція `citric_acid` з `fixed_acidity` дорівнює 0.67. Заміна

`fixed_acidity` на наступну за розкидом `density` дає набір, де найбільший модуль кореляції становить 0.50 замість 0.67.

Межі `vmin=-1, vmax=1` задаються явно. Без них шкала розтягнеться по фактичному діапазону, нуль перестане бути серединою палітри, і кореляція 0.2 виглядатиме такою ж насиченою, як 0.9.

Проєкція на дві головні компоненти - остання лінія для 8+ ознак.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
xy = pca.fit_transform(sample4[features4])
print(pca.explained_variance_ratio_.round(3))

fig, ax = plt.subplots(figsize=(6, 5))
for value in sorted(sample4["class"].unique()):
    mask = (sample4["class"] == value).to_numpy()
    ax.scatter(xy[mask, 0], xy[mask, 1], s=8, label=f"клас {value}")
ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
ax.legend()
fig.savefig(FIG / "sample4_pca.png", dpi=120, bbox_inches="tight")
```

PCA стискає будь-яку кількість ознак у дві осі, тому вибірка з одинадцятьма ознаками поміщається на одну площину. Плата за це - осі перестають бути ознаками: `PC1` є лінійною комбінацією всіх одинадцяти, і твердження на кшталт «кластери розділяються за вмістом алкоголю» з такого графіка не випливає.

Разом з графіком наводиться `explained_variance_ratio_` - частка розкиду, що вціліла в площині. На ірисах це 0.841 + 0.117 = 0.958, тобто площина зберігає майже все. На `wine-quality-red` - 0.358 + 0.194 = 0.552, майже половина мінливості лишилася поза екраном, і злиті на такій проєкції класи цілком можуть виявитися розділеними в повному просторі. Тому PCA лишається доповненням до матриці розсіювання, а не заміною їй.

Мінімум для звіту. По кожній вибірці - щонайменше один графік, на якому видно розділення на кластери, з підписаними осями, легендою за міткою і копією у `figures/`. Для Вибірок 3 і 4 одна площина x-у цю умову не виконує.

---

5. Вибірка 1: намальована мишею

Постановка задачі:

- Власноруч створити вибірку із лінійно-сепарабельними кластерами (від 3 до 5 кластерів)
- Елемент вибірки має ознаки: координати x, y, а також номер кластера
- Розмір вибірки від 1000 елементів
- Встановити та використати пакет `drawdata` (https://github.com/koaning/drawdata)
- Нормалізувати дані
- Візуалізувати дані
- Експортувати нормалізовану вибірку до csv-файлу

5.1. Порядок дій

1. Створити віджет: `ScatterWidget` з пакета `drawdata`; параметри `width`, `height`, `brushsize` задаються при створенні. Віджет повернути останнім виразом клітинки, щоб він відобразився.
2. Намалювати згустки мишею: колір вибирається в палітрі віджета, кожен кластер - свій колір.
3. **В окремій наступній клітинці** зчитати дані властивістю `widget.data_as_pandas`. Повертається таблиця з колонками `x`, `y`, `color`, де `color` - hex-рядок.
4. Перетворити колір на цілий номер кластера (`pandas.factorize`), перейменувати колонку.
5. Нормалізувати ознаки `x`, `y`, зберегти у `data/sample1.csv`, побудувати графік.

5.2. Перевірка (опціонально)

```
print(sample1.shape)                              # рядків від 1000, колонок 3
print(sample1.cluster.value_counts().sort_index())  # 3-5 кластерів, розміри співмірні
print(sample1[["x", "y"]].agg(["min", "max"]))    # 0.0 та 1.0
```

5.3. Рекомендації (опціонально)

- **Лінійно-сепарабельні** означає, що між кластерами можна провести пряму так, щоб жодна точка не опинилася по чужий бік. Згустки не повинні дотикатися. Якщо два кольори змішалися - стерти і перемалювати, у ЛР2 така вибірка дасть неінформативний результат.
- **1000 точок - це багато для однієї спроби.** Простіше малювати довгими повільними мазками, ніж короткими рухами. Розмір пензля `brushsize` задається при створенні віджета.
- **Розподіляти точки між кластерами приблизно порівну.** Кластер зі 100 точок проти кластера з 900 в ЛР2 регулярно розчиняється у сусідньому.
- Якщо на графіку картинка виглядає перевернутою по вертикалі - це нормально: віджет віддає екранні координати, у яких вісь y росте вниз. За потреби вісь перевертається після нормалізації: `1 - y`.

> **Типова помилка.** Замість полотна для малювання у клітинці з'являється повідомлення `Failed to load model class 'AnyModel' from module 'anywidget'`, а в консолі браузера - рядок виду `Module anywidget, version ~0.9.* is not registered, however, 0.11.0 is`. Віджет `drawdata` складається з двох половин: Python-частина працює в ядрі, JavaScript-частина - у вкладці браузера. Обидві дає пакет `anywidget`, і номери їхніх версій мають збігатися. У повідомленні зліва стоїть версія, яку просить ядро, справа - версія, завантажена сторінкою; помилка означає, що половини розійшлися. Порядок дій:
>
> 1. `uv sync` з теки проєкту - повертає `.venv` до версій із `uv.lock`;
> 2. зупинити JupyterLab (`Ctrl+C` у терміналі) і запустити його наново командою `uv run jupyter lab`;
> 3. перезавантажити вкладку з очищенням кеша - `Ctrl+Shift+R`;
> 4. `Kernel` -> `Restart Kernel` і виконати клітинку з віджетом повторно.
>
> Кроки 2 і 3 пропускати не можна: JavaScript-частина завантажується у сторінку один раз при її відкритті, тому вкладка, відкрита до `uv sync`, і далі тримає стару версію. Закріплювати версію `anywidget` вручну не треба і шкідливо: потрібну версію добирає `drawdata`, а ручне закріплення дає рівно цю саму помилку.
>
> **Типова помилка.** `widget.data_as_pandas` повертає порожню таблицю з нульовою кількістю колонок. Це означає, що клітинка з читанням даних виконалася до того, як щось намальовано. Треба намалювати, а потім виконати клітинку читання повторно. Читати дані у тій самій клітинці, де створюється віджет, не можна.

---

6. Вибірка 2: згенерована

Постановка задачі:

- Реалізувати алгоритм генерації випадкової вибірки із параметрами:
  - K - кількість кластерів
  - N_k - кількість елементів у кластері
  - Відрізок [σ1, σ2] - діапазон значень можливої дисперсії нормального розподілу
- Алгоритм вибирає K центрів (математичне сподівання нормального розподілу) у тривимірному просторі та випадкове значення дисперсії із відрізку [σ1, σ2], та генерує N_k точок для кожного кластеру за відповідним нормальним розподілом
- Елемент вибірки має ознаки: координати x, y, z, а також номер кластера
- Нормалізувати дані
- Візуалізувати дані
- Вибрати та запам'ятати параметри генерації, експортувати нормалізовану вибірку до csv-файлу

Бажано, щоб кластери трохи перетиналися. Вибірка 1 за умовою лінійно-сепарабельна, тому k-means, Forgy, Random partition і k-means++ дадуть на ній той самий поділ, і порівнювати їх буде ні за чим. Якщо рознести і згустки Вибірки 2 так, що між ними порожньо, ЛР2 знову покаже 100% збігу з відомими мітками. Перетин у 2-8% лишає кілька десятків прикордонних точок, які різні ініціалізації відносять до різних кластерів, а метод ліктя і силуетний аналіз дають нетривіальну відповідь.

6.1. Контракт функції

```
def generate_sample(K, N_k, sigma_range, seed=None, center_range=(0.0, 100.0)):
    """
    K            - кількість кластерів
    N_k          - кількість точок у кожному кластері
    sigma_range  - кортеж (sigma_1, sigma_2), діапазон дисперсії
    seed         - зерно генератора для відтворюваності
    center_range - межі, в яких випадково вибираються центри

    Повертає (DataFrame з колонками x, y, z, cluster; centers; sigmas)
    """
```

Потрібні засоби: `numpy.random.default_rng` - генератор; його методи `uniform` (центри і дисперсії) та `normal` (точки кластера, `size=(N_k, 3)`); `pandas.DataFrame` і `concat` для збирання результату.

6.2. Перевірка структури (опціонально)

```
raw2, centers, sigmas = generate_sample(K=4, N_k=300, sigma_range=(11.0, 15.0), seed=2026)

assert raw2.shape == (4 * 300, 4), raw2.shape
assert list(raw2.columns) == ["x", "y", "z", "cluster"]
assert raw2.cluster.nunique() == 4
assert raw2.groupby("cluster").size().nunique() == 1   # кластери однакового розміру
```

6.3. Перевірка ступеня перетину (опціонально)

На тривимірному графіку перетин на око не оцінити, тому його рахують числом: відсоток точок, що лежать ближче до чужого центру, ніж до власного. Це ті самі точки, які ЛР2 покаже як віднесені не до свого кластера.

```
from itertools import combinations

def cluster_report(df, centers, sigmas, cols=("x", "y", "z")):
    X = df[list(cols)].to_numpy()
    d = np.linalg.norm(X[:, None, :] - centers[None, :, :], axis=2)
    overlap = 100.0 * (d.argmin(axis=1) != df["cluster"].to_numpy()).mean()

    pairs = list(combinations(range(len(centers)), 2))
    d_min = min(np.linalg.norm(centers[i] - centers[j]) for i, j in pairs)
    sep = min(np.linalg.norm(centers[i] - centers[j]) / (sigmas[i] + sigmas[j])
              for i, j in pairs)

    print(f"мін. відстань між центрами: {d_min:.1f}")
    print(f"коефіцієнт роздільності:    {sep:.2f}  (цільовий діапазон 1.5-2.0)")
    print(f"перетин кластерів:          {overlap:.1f}%  (цільовий діапазон 2-8%)")
    return overlap, sep

cluster_report(raw2, centers, sigmas)
```

Для наведеного виклику вивід такий:

```
мін. відстань між центрами: 47.2
коефіцієнт роздільності:    1.71  (цільовий діапазон 1.5-2.0)
перетин кластерів:          5.2%  (цільовий діапазон 2-8%)
```

Коефіцієнт роздільності - це мінімальна по всіх парах кластерів величина `d(центр_i, центр_j) / (σ_i + σ_j)`. Вона і керує перетином:

Коефіцієнт	Перетин	Що вийде в ЛР2
понад 3.0	0%	усі алгоритми дають ідентичний результат, порівнювати нічого
2.0-3.0	0-3%	перетин майже непомітний
1.5-2.0	**2-8%**	**цільовий діапазон**
1.0-1.5	8-20%	межа розмивається, метод ліктя дає нечіткий злам
менше 1.0	понад 20%	кластери зливаються, вибірка непридатна

Як гарантовано влучити в діапазон. Центри вибираються випадково, тому фіксований `sigma_range` при одному зерні дасть 0% перетину, а при іншому - 30%. Надійніше прив'язати σ до фактичної відстані між центрами: спочатку вибрати центри, обчислити `d_min`, потім узяти `sigma_range = (0.25·d_min, 0.33·d_min)`. Це дає коефіцієнт роздільності 1.6-2.0 незалежно від зерна. Перевірено на шести зернах: перетин склав від 2.0% до 6.5%.

Якщо `cluster_report` показав число поза діапазоном - не перегенеровувати наосліп, а змінити `sigma_range` у потрібний бік: більший перетин потребує більшої σ.

6.4. Збереження і візуалізація

Нормалізована вибірка зберігається у `data/sample2.csv`.

Постановка задачі вимагає також запам'ятати параметри генерації. Спосіб на вибір:

- лишити в ноутбуці рядок виклику `generate_sample` з усіма аргументами і зафіксованим `seed`. Цього досить, щоб відтворити ті самі центри, дисперсії й точки;
- або додатково вивантажити центри, дисперсії, K і N_k окремим файлом, наприклад `data/sample2_params.csv`. Зручно, якщо параметри доведеться підбирати кілька разів і треба бачити історію.

Вимога виконана в обох випадках. Що саме вибрано, зазначається у звіті.

Для тривимірного графіка потрібна вісь з `projection="3d"` (`Figure.add_subplot`), точки наносяться по кластерах у циклі. Ракурс на такому графіку приховує частину структури, тому його доповнюють матрицею розсіювання 3×3 (розділ 4.1): вона дає проєкції x-y, x-z, y-z одним викликом.

6.5. Рекомендації (опціонально)

- **Задавати `seed`.** Без нього кожен перезапуск ноутбука дає іншу вибірку, і числа у звіті перестають відповідати графікам. Зерно фіксує генерацію повністю.
- **Використовувати `np.random.default_rng(seed)`, а не `np.random.seed`.** Другий варіант працює з глобальним станом, який змінюють будь-які сторонні виклики, тому відтворюваність не гарантується.
- **Значення `cluster_report` навести у звіті.** Вони знадобляться в ЛР2 як точка відліку, з чим порівнювати результат алгоритму.
- **Перевіряти, що центри не збіглися.** При випадковому виборі два центри можуть опинитися поруч, і тоді два кластери зіллються в один незалежно від σ. Захист - перевірка мінімальної

попарної відстані: якщо `d_min` менша за чверть розміру куба `center_range`, центри варто перегенерувати.

- **Візуально перевіряти перетин на двовимірних проєкціях, а не на 3D-графіку.** На тривимірному графіку згустки перекриваються через ракурс навіть тоді, коли в просторі вони роз'єднані. Проєкції x-y, x-z, y-z показують картину чесніше, а `scatter_matrix` будує всі три одразу - фрагмент у розділі 4.1.
- Розмір вибірки має бути від 1000 елементів, тобто K × N_k ≥ 1000.

---

7. Вибірка 3: іриси Фішера

Постановка задачі:

- Завантажити csv-файл вибірки «Іриси Фішера» до простору JupyterLab
- Нормалізувати дані
- Візуалізувати дані

Файл `дані/iris.csv` покласти в `data/` проєкту. У тій самій теці лежать `iris.xlsx` з тими самими даними і `iris.pdf` з описом набору. Для роботи потрібен саме csv.

7.1. Порядок дій

1. Прочитати csv (`pandas.read_csv`). Колонки: `Sepal_length`, `Sepal_width`, `Petal_length`, `Petal_width`, `Species`.
2. Перетворити `Species` на цілий номер кластера (`pandas.factorize`), зберігши при цьому вихідну текстову колонку.
3. Нормалізувати чотири числові ознаки, зберегти у `data/sample3.csv`.
4. Побудувати матрицю розсіювання: `pandas.plotting.scatter_matrix` з `c=` за номером кластера, готовий фрагмент разом з легендою - у розділі 4.1. Одна площина x-y чотиривимірну вибірку не показує, а матриця дає всі пари ознак одразу.

7.2. Перевірка (опціонально)

```
print(iris.shape)                          # (150, 5)
print(iris.isna().sum().sum())             # 0
print(sample3.groupby("Species").cluster.first())   # три різні номери
```

7.3. Рекомендації (опціонально)

- **У звіті описати, що видно на матриці.** `Iris-setosa` відокремлюється від решти за довжиною і шириною пелюстки, `versicolor` і `virginica` частково перекриваються. Це пояснить результати ЛР2, де два кластери зіллються.
- **Мітку `Species` лишати в csv.** Вона знадобиться в ЛР2 для порівняння результату кластеризації з відомим значенням.
- Якщо береться `iris.xlsx`, читати через `pandas.read_excel` з `engine="openpyxl"` і перевірити назви колонок - вони можуть відрізнятися від csv-варіанта.

---

8. Вибірка 4: реальні дані

Постановка задачі:

- Вибрати (знайти) вибірку на основі реальних даних, що містить розділення на кластери:
  - https://www.kaggle.com/datasets?tags=13304-Clustering
  - https://huggingface.co/scikit-learn/datasets
  - інші джерела
- Вибірка має бути унікальною в межах підгрупи навчання
- Розмір вибірки від 1000 елементів
- Вибрати від 5 характерних ознак, а також Label (або номер кластеру)
- Прибрати викиди з вибірки (значення, що значно відрізняються від інших значень елементів вибірки), за можливості заповнити пропущені значення усередненнями (або видалити такі елементи) та нормалізувати дані
- Візуалізувати дані
- Експортувати нормалізовану вибірку до csv-файлу

Найшвидший спосіб без ручного завантаження - `sklearn.datasets.fetch_openml` з `as_frame=True`: набір повертається готовим `DataFrame` у полі `.frame`. Приклади в цьому розділі виміряні на наборі `wine-quality-red` (1599 рядків, 11 числових ознак, мітка `class`). Вибирати треба свій - вимога унікальності в межах підгрупи лишається.

8.1. Огляд перед обробкою (опціонально)

```
print(df4.shape)
print(df4.dtypes)
print(df4.isna().sum())
df4.describe().T
```

На що дивитися в `describe()`:

- `max` на порядок більший за 75-й процентиль - ознака викидів;
- від'ємні значення там, де їх бути не може;
- `std`, що дорівнює нулю - константна ознака, її треба прибрати, бо в нормалізації вона дасть ділення на нуль.

8.2. Пропущені значення

Порядок: спочатку викинути рядки, у яких бракує занадто багато ознак (`DataFrame.dropna` з параметрами `subset` і `thresh`), потім заповнити поодинокі пропуски (`DataFrame.fillna`). Кількість рядків до і після вивести числом.

Заповнювати медіаною, а не середнім. Середнє саме зміщується викидами, які ще не прибрані, тому заповнення середнім втягує їхній вплив у решту рядків.

8.3. Викиди

Правило міжквартильного розмаху: викид - це значення поза межами [Q1 - k·IQR, Q3 + k·IQR], де `k` за замовчуванням 1.5. Квартилі рахуються методом `DataFrame.quantile`, межі - поелементно по

колонках.

Рядок з викидом можна видалити (маска по всіх ознаках через `.any(axis=1)`) або обрізати його значення до меж (`DataFrame.clip` з `axis=1`).

> **Типова помилка, яку треба перевірити числом.** Видалення за умовою `.any(axis=1)` викидає рядок, якщо він виходить за межі хоча б за однією ознакою з одинадцяти. Чим більше ознак, тим більше рядків потрапляє під ніж. На наборі `wine-quality-red` це прибирає 405 рядків з 1599, тобто 25.3% вибірки. Орієнтир - не більше 10%.

Втрата понад 10% означає, що розподіл ознак далекий від нормального і правило 1.5·IQR до нього не застосовне. Дані при цьому можуть бути цілком чистими. Варіанти, виміряні на цьому ж наборі:

Варіант	Результат
видалення, `k = 1.5`	1599 -> 1194, прибрано 25.3%
видалення, `k = 2.0`	1599 -> 1340, прибрано 16.2%
видалення, `k = 3.0`	1599 -> 1435, прибрано 10.3%
обрізання, `k = 1.5`	1599 -> 1599, підрізано значення у 25.3% рядків
правило лише до ознак, де викиди дійсно є	залежить від набору

Обрізання зберігає всі 1599 рядків, тому при одинадцяти ознаках підходить краще за видалення. Скільки рядків при цьому змінилося, рахується порівнянням таблиці до і після `clip`.

Вибраний варіант і його наслідки треба назвати у звіті числом «було - стало». Просто лишити 25% втрати без пояснення не можна.

8.4. Нормалізація, експорт, візуалізація

Нормалізуються лише числові ознаки, мітка лишається як є. Результат - у `data/sample4.csv`.

Матриця розсіювання по одинадцяти ознаках нечитабельна (121 клітинка), тому ознаки спочатку відбираються, а вже потім будується матриця по чотирьох-п'яти з них. Відбір у два кроки:

1. Взяти ознаки з найбільшим стандартним відхиленням (`DataFrame.std` з сортуванням) - константна ознака кластерів не розділяє.
2. Перевірити відібране за теплокартою кореляцій (розділ 4.1): якщо в парі модуль кореляції перевищує 0.6, одна з двох ознак дублює іншу, і замість неї береться наступна за розкидом.

Матриця розсіювання з розфарбуванням за міткою - основний графік Вибірки 4. Ознаки, що не потрапили в матрицю, показуються одним зі зведених графіків розділу 4.1: паралельні координати по підвибірці або проєкція на дві головні компоненти. Який саме відбір застосовано і які ознаки лишилися, зазначається у звіті.

8.5. Рекомендації (опціонально)

- **Перевірити, що Label справді існує.** Набір без розділення на класи не задовольняє умові і зробить неможливим пункт ЛР2 про порівняння з відомим значенням.
- **Категоріальні ознаки не нормалізуються.** Рядкові колонки або кодуються (`pandas.get_dummies`), або виключаються зі списку ознак.

- **Ідентифікатори прибрати.** Колонки виду `id`, `index`, `Unnamed: 0` числові, але змісту не несуть і зіпсують відстань у ЛР2.
- **Зберігати також вихідний файл.** Якщо обробка виявиться надто агресивною, буде з чого почати заново.

---

9. Фінальна перевірка (опціонально)

Перед написанням звіту виконати в ноутбуці:

```
SPEC = {
    "sample1": ["x", "y"],
    "sample2": ["x", "y", "z"],
    "sample3": ["Sepal_length", "Sepal_width", "Petal_length", "Petal_width"],
    "sample4": num,          # список ознак Вибірки 4 з розділу 8.2
}

for name, features in SPEC.items():
    df = pd.read_csv(DATA / f"{name}.csv")
    f = df[features]
    print(f"{name}: {df.shape[0]} рядків, ознак {len(features)}, "
          f"NaN={int(df.isna().sum().sum())}, "
          f"min={f.min().min():.3f}, max={f.max().max():.3f}")
```

Очікуваний вигляд виводу:

```
sample1: 1200 рядків, ознак 2, NaN=0, min=0.000, max=1.000
sample2: 1200 рядків, ознак 3, NaN=0, min=0.000, max=1.000
sample3: 150 рядків, ознак 4, NaN=0, min=0.000, max=1.000
sample4: 1599 рядків, ознак 11, NaN=0, min=0.000, max=1.000
```

У кожній вибірці немає жодного NaN, ознаки лежать у [0, 1], вибірки 1, 2 і 4 мають від 1000 рядків.

> **Типова помилка.** Спокуса написати коротше - `num = df.select_dtypes("number")` замість явного списку ознак. Так робити не можна: після `read_csv` колонка `cluster` (значення 0-3) і мітка Вибірки 4 (значення 3-8) читаються як звичайні цілі числа й потрапляють у вибірку колонок. Перевірка тоді покаже `max=3.000` і `max=8.000`, і буде незрозуміло, це зламана нормалізація чи мітка в кадрі. Список ознак треба задавати явно.

---

10. Звіт

Постановка задачі. Звіт з лабораторної роботи має містити:

1. Титульна сторінка
2. Постановка задачі
3. Хід виконання
    - Описати процес налаштування середовища
    - Показати характерні елементи кожної вибірки

- Навести код ключових кроків разом з результатами, які він дав
- Описати кроки, специфічні для взятого джерела даних: відбір ознак, виявлення викидів, обробку пропущених значень
- Візуалізувати вибірки на графіках/діаграмах

4. Висновок

Що це означає на практиці:

- **Постановка задачі** - з розділу 0 або з файлу постановки задачі.
- **Налаштування середовища** - які команди виконано, яка версія Python, які пакети і навіщо.
- **Характерні елементи вибірки** - перші 5-10 рядків таблиці для кожної з чотирьох.
- **Графіки** - усі чотири вибірки. Для Вибірок 3 і 4 площини x-y недостатньо, потрібна матриця розсіювання або інший графік з розділу 4.1.
- **Код** - фрагментами, не всім ноутбуком: функція генерації Вибірки 2, виявлення і обробка викидів Вибірки 4, нормалізація, побудова графіків. Кожен фрагмент супроводжується числом або графіком, який він дав.
- **Кроки, специфічні для взятого джерела** - те, чого немає в цьому документі, бо залежить від конкретного набору даних. Саме ця частина показує роботу, і саме її неможливо переписати з методички.

Що входить у специфічні кроки для Вибірки 4:

Крок	**Що навести у звіті**
джерело	звідки взято набір, скільки рядків і колонок було спочатку
відбір колонок	які колонки відкинуто (ідентифікатори, категоріальні, константні) і чому
пропуски	скільки їх було по колонках, що зроблено - видалення чи заповнення, чим саме
викиди	яким правилом виявлено, з яким `k`, які межі вийшли, скільки рядків зачеплено
відбір ознак для графіка	за яким критерієм вибрано ознаки для матриці розсіювання

Формулювання про викиди має бути перевіряним числом, а не словом «прибрано». Зразок рівня деталізації: «Межі 1.5·IQR по одинадцяти ознаках; видалення рядків за умовою `.any(axis=1)` прибирало 405 рядків з 1599 (25.3%), що перевищує орієнтир 10%, тому застосовано обрізання `clip`: усі 1599 рядків збережено, значення підрізано у 25.3% рядків».

Висновок - про результат роботи, а не про її перебіг. Результат тут один: чотири вибірки, готові до ЛР2, і знання про те, чого від кожної чекати. Звідси чотири пункти:

- що вийшло: розмір, кількість ознак і кількість кластерів кожної вибірки після нормалізації;
- чим вони відрізняються за складністю для кластеризації і з чого це видно на графіках;
- які параметри генерації Вибірки 2 вибрано, який вийшов перетин, і скільки рядків втрачено на очищенні Вибірки 4 - обидва числа знадобляться в ЛР2 як точка відліку;
- який пункт виявився найпроблемнішим і як його розв'язано.

Переказ виконаних дій у висновок не входить, для нього є хід виконання.

Формат: `.doc`, `.docx`, `.odt` або `.pdf`.

Ноутбук не замінює звіт. Ноутбук показує, що код працює; звіт показує розуміння результату.

---

11. Здача

До завдання додаються:

1. Архів з проєктом: ноутбук, `figures/` і файли оточення, якщо вони є (`pyproject.toml` з `uv.lock` , `requirements.txt`). Теку оточення (`.venv/`) в архів **не класти** - вона відновлюється командою `uv sync` або `pip install -r` .
2. Архів з чотирма csv-файлами вибірок. Якщо параметри генерації Вибірки 2 збережено окремим файлом, покласти його сюди ж.
3. Звіт.

Для повної оцінки роботи захист голосом у просторі Teams є необхідним.

---

12. Чек-лист

#	Пункт	Перевірка
1	JupyterLab запускається, пакети з розділу 2.1 імпортуються	клітинка з імпортами виконується без помилок
2	Вибірка 1: 3-5 кластерів, ≥1000 точок, не перетинаються	`value_counts()` + графік
3	Вибірка 2: K × N_k ≥ 1000, параметри генерації запам'ятані	`seed` і аргументи виклику в ноутбуці або окремий файл параметрів
3а	Вибірка 2: перетин кластерів у межах 2-8% (опціонально)	вивід `cluster_report`
4	Вибірка 3: 150 рядків, 3 класи, мітка збережена	`shape` та `nunique()`
5	Вибірка 4: ≥1000 рядків, ≥5 ознак, є Label, втрати ≤10%	вивід «було -> стало»
6	Усі ознаки в [0, 1], NaN немає	цикл з розділу 9
7	Мітки кластерів не нормалізовані	значення лишилися цілими
8	Чотири графіки збережені у `figures/`	`ls figures`
9	Вибірки 3 і 4 показані матрицею розсіювання (або графіком з розділу 4.1)	`ls figures`
10	Звіт містить титульну сторінку, постановку задачі, хід виконання і висновок	-
11	Звіт містить код фрагментами і специфічні кроки по Вибірці 4 з числами	таблиця з розділу 10
12	Висновок описує результат: що дала кожна вибірка і чого чекати від неї в ЛР2	чотири пункти з розділу 10, з числами